

Problem Solving

Learning Objectives

- Apply systematic problem-solving strategies to programming challenges
- Decompose problems into subproblems and identify patterns
- Translate real-world scenarios into computational solutions

Engage (5-7 minutes)

A doctor does not prescribe medicine immediately—first, they diagnose by asking questions, examining symptoms, and eliminating possibilities. Problem solving in programming follows the same approach: understand the problem, plan a strategy, implement the solution, and verify the results. Today we sharpen our ability to tackle any programming challenge methodically.

Explore (10-12 minutes)

Take this problem: “Given a string of parentheses like ‘`()()`’, determine if they are balanced.” First, discuss what “balanced” means with a partner. List test cases: “`()`”, “`()`”, “`()`”, “`()()`”. Try to solve it manually for each case. Can you identify a pattern in how balanced parentheses work? Write down your observations before writing any code.

Explain (10-12 minutes)

Effective problem solving follows a structured approach: understand the problem thoroughly, identify inputs and outputs, break the problem into smaller subproblems, look for patterns, design a solution, code it, and test it with edge cases. Common strategies include brute force (trying all possibilities), divide and conquer, greedy algorithms (making locally optimal choices), and dynamic programming (storing solutions to subproblems). Debugging skills and testing are integral parts of the process.

Elaborate (8-10 minutes)

Solve these problems step by step: reverse a linked list, find all pairs in an array that sum to a target, and implement a simple calculator that handles operator precedence. For each, first write the problem in your own words, identify edge cases, design the algorithm in pseudocode, then implement and test it. Compare your solutions with a partner and discuss which approaches are most elegant and efficient.

Evaluate (5-8 minutes)

Given the problem “find the longest consecutive sequence in an unsorted array,” write a complete solution including: problem definition, input/output examples, edge cases, algorithm design in pseudocode, and the working code. Reflect on which problem-solving strategy you used and why.